Static Structure Factor

Chi Y. Cheng, Maciej Bartkowiak, Sanghamitra Mukhopadhyay

MDANSE Workshop 2026

04/11/2026



Static structure factor

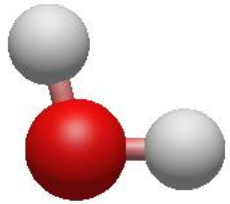
- In this workshop exercise, we will run simulation of water using OpenMM and then run static structure factor calculations with MDANSE.
- Using a PDB file we will use Packmol to generate a box of water molecules.
- Actually there is a function in OpenMM method `Modeller.addSolvent` which we could use to solvate an empty box with water.
- We will use Packmol as it is useful to learn how to use this program since you may want to create a box of other molecules or a mixture of different molecules in the future.

OpenMM



Packmol

- If the files you should see `water.pdb`. This is a PDB file containing a single water molecule.



		atom name	residue name			x position	y position	z position			element
ATOM	1	O	HOH	A	1	0.000	0.000	0.000	1.00	0.00	O
ATOM	2	H1	HOH	A	1	0.957	0.000	0.000	1.00	0.00	H
ATOM	3	H2	HOH	A	1	-0.240	0.927	0.000	1.00	0.00	H
END											

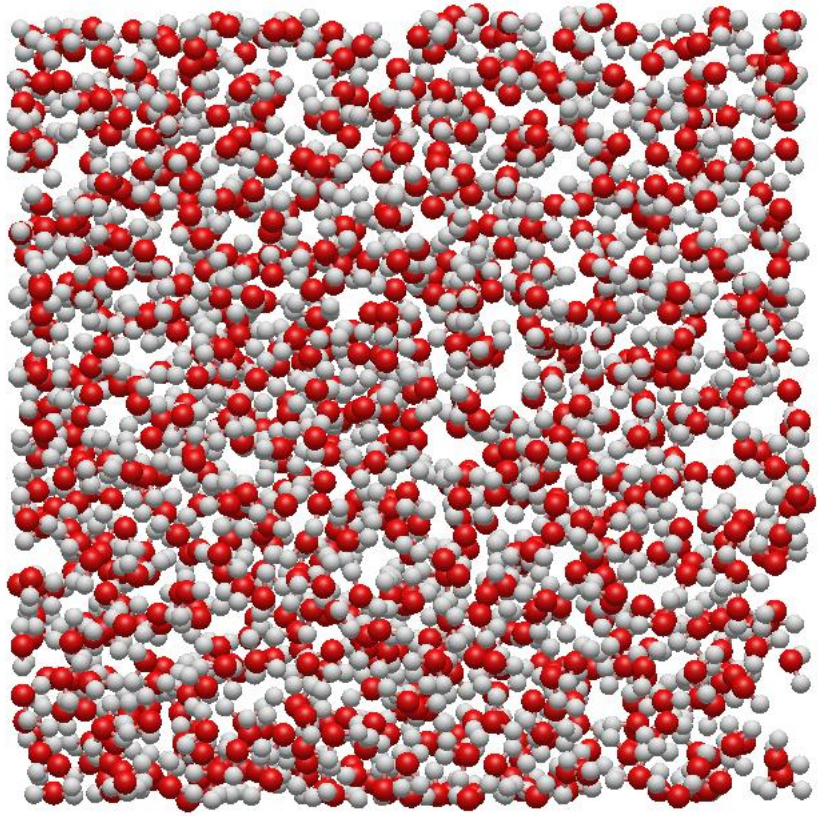
Packmol

- First let's install a few thing in a new conda environment.

```
conda create -n openmm python=3.13
conda activate openmm
pip install packmol
pip install openmm
pip install pandas
pip install matplotlib
```

- On windows you'll need `libgfortran-5.dll` you can install this with.

```
conda install -c conda-forge libgfortran5
```



Packmol

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with Packmol to generate a box of water molecules.

```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```

Packmol

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with Packmol to generate a box of water molecules.
- Tolerance the minimum distance between atoms in angstrom.



```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```

Packmol

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with Packmol to generate a box of water molecules.
- Tolerance the minimum distance between atoms in angstrom.
- Output file type and name.



```
tolerance 2.0
filetype pdb
output water_box.pdb

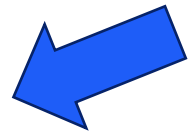
structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```

Packmol

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with Packmol to generate a box of water molecules.
- Tolerance the minimum distance between atoms in angstrom.
- Output file type and name.
- Input file for one of the molecules. Here we have water and specified that we want 1000 of them.

```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```



Packmol

- `inside box` specifies the minimum and maximum xyz coordinates to place the molecules.
- Note that we have left a border where there will be no atoms. This is because Packmol does not have periodic boundaries while our MD simulation will. This will stop atoms from overlapping over each other when we apply periodic boundaries.
- The empty border will get filled in when we run our MD simulation.

```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```



Packmol

- You can run PACKMOL with this command. If you are using windows, make sure you are using cmd.

```
packmol < water_packmol.inp
```

- If it all runs correctly you should see this success message.

```
###############################################################################

                                  Success!
                 Final objective function value: .42726E+00
                 Maximum violation of target distance:   0.000000
                 Maximum violation of the constraints: .83285E-02

-------------------------------------------------------------------------------

                 Please cite this work if Packmol was useful:

             L. Martinez, R. Andrade, E. G. Birgin, J. M. Martinez,
           PACKMOL: A package for building initial configurations for
                      molecular dynamics simulations.
          Journal of Computational Chemistry, 30(13) pp. 2157-2164, 2009.
                    https://doi.org/10.1002/jcc.21224

###############################################################################

   Running time:    0.703125000      seconds.

-------------------------------------------------------------------------------
```

OpenMM

- Now we have a box of water molecules in the PDB file format.
- In the files there is a python script to run OpenMM with your PDB file. However, the script is not runnable and needs to be completed.
- If you see … this is where you will need to fill in to complete the script.

```python
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- We will run simulation on light water, heavy water, and null water. We will need to change these later but for now we will be running the simulation for light water.

```python
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- We will run simulation on light water, heavy water, and null water. We will need to change these later but for now we will be running the simulation for light water.
- This script will output two files, a DCD file and PDB file which we will use with the DCD converter to generate MDANSE trajectory files.
- Since we are generating three different trajectory set a good output name for them with the extension .dcd and .pdb.

```python
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- Here we have specified the temperature in units of kelvin. We want to run our simulations at room temperature and pressure.

```python
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- Here we have specified the temperature in units of kelvin. We want to run our simulations at room temperature and pressure.
- For the time step should depend on the water model you want to use. If you want to run a flexible water model this should be small around 0.5 fs or smaller, for a rigid water model you can use a timestep of 2.0 fs or smaller.

```python
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- In the next block we specify the number of time steps to run a simulation with 50, 25, and 10 ps for the NPT, NVT, and NVE equilibration stages and 100 ps for the NVE production.

```python
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 /  value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 /  TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- In the next block we specify the number of time steps to run a simulation with 50, 25, and 10 ps for the NPT, NVT, and NVE equilibration stages and 100 ps for the NVE production.
- In this line we specify the report interval this will specify how often we will output the MD simulation information.

```python
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAL = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- In the next block we specify the number of time steps to run a simulation with 50, 25, and 10 ps for the NPT, NVT, and NVE equilibration stages and 100 ps for the NVE production.
- In this line we specify the report interval this will specify how often we will output the MD simulation information.
- Here we define the DCD interval which we use to tell the reporter how often to dump the trajectory results. We will do this every 20 fs.

```
import sys

import numpy as np
import openmm as mm
import openmm.app as app
import openmm.unit as unit


rng = np.random.default_rng()

LIGHT_WATER = True
D_FRACTION = 0.0

PACKED_PDB = "water_box.pdb"
OUTPUT_DCD_NAME = ...
OUTPUT_PDB_NAME = ...

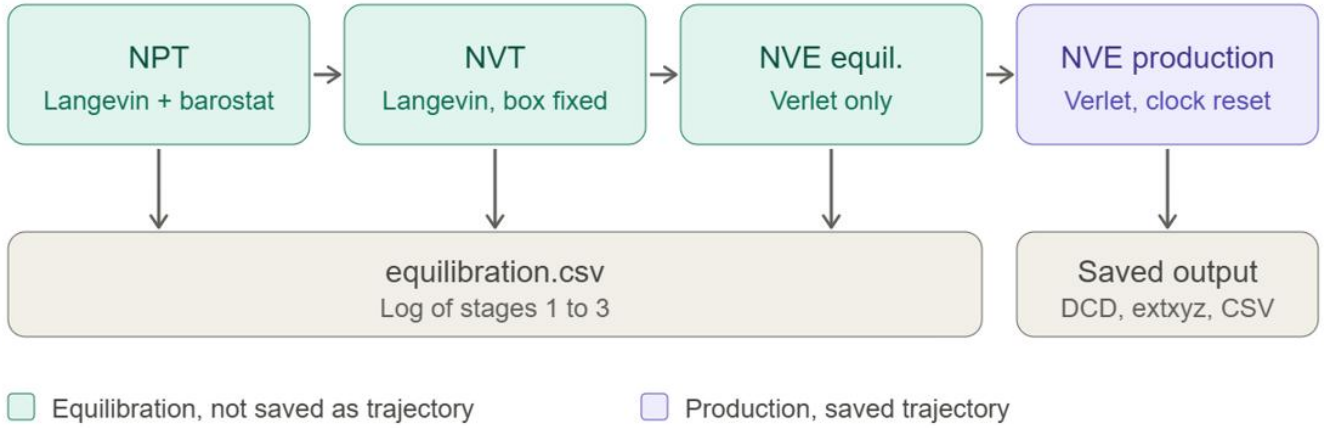
TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAL = ...(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))

H_MASS = 1.00782503 * unit.amu
D_MASS = 2.01410178 * unit.amu
```

OpenMM

- So why are we running MD like this? In the next tutorials we will want to run distinct van-Hove function calculations which calculates distances between atoms at different times.
- A barostat can cause the distances an atom has travel over time can be distorted by unit cell fluctuations. Similarly, a thermostat can interfere with the atom velocities and therefore its trajectory. Remember that NPT/NVT is about generating the right sampling for there respective ensembles, the trajectories themselves will be distorted in some way.
- We therefore must use NVE for production. We run NPT to equilibrate the density, then fix the volume and equilibrate the temperature only with NVT, then a finally an NVE equilibration to prepare it ready for the NVE production.



OpenMM

- Here we have a function which randomly changes the water molecules in our simulation to a specific fraction of D2O, HDO, and H2O (protium water).

```python
def transmute_water(openmm_system, openmm_topology, d_fraction):  1 usage
    hydrogens = [a for a in openmm_topology.atoms() if a.element == app.element.hydrogen]
    n_d = int(round(d_fraction * len(hydrogens)))
    chosen = rng.choice(len(hydrogens), size=n_d, replace=False).tolist()
    d_atoms = {hydrogens[k].index for k in chosen}
    names = {0: "HOH", 1: "HDO", 2: "DOD"}
    counts = {0: 0, 1: 0, 2: 0}
    for residue in openmm_topology.residues():
        n_h = 0
        n_deut = 0
        for atom in residue.atoms():
            if atom.element != app.element.hydrogen:
                continue
            n_h += 1
            if atom.index in d_atoms:
                n_deut += 1
                atom.name = f"D{n_h}"
                atom.element = app.element.deuterium
                openmm_system.setParticleMass(atom.index, D_MASS)
            else:
                openmm_system.setParticleMass(atom.index, H_MASS)
        residue.name = names[n_deut]
        counts[n_deut] += 1
    print(f"{n_d} of {len(hydrogens)} hydrogens are deuterium: "
          f"{counts[2]} D2O, {counts[1]} HDO, {counts[0]} H2O")
```

OpenMM

- In the next part we load our PDB file and specify the size of our simulation cell.

```
60  # load the pdf file and set the box vectors
61  pdb = app.PDBFile(PACKED_PDB)
62  topology = pdb.topology
63  topology.setPeriodicBoxVectors(
64      [mm.Vec3(3.105, 0, 0),
65       mm.Vec3(0, 3.105, 0),
66       mm.Vec3(0, 0, 3.105)] * unit.nanometer
67  )
68
69  # create the force field and add extra particles
70  forcefield = app.ForceField(...)
71  modeller = app.Modeller(pdb.topology, pdb.positions)
72  modeller.addExtraParticles(forcefield)
73  topology = modeller.topology
```

OpenMM

- In the next part we load our PDB file and specify the size of our simulation cell.
- Here, we specify the force field. Here you will choose the force field that you want to use. In OpenMM force fields are defined using an XML file. This contains all the force field parameters.
- You can give OpenMM your own custom force field parameter file but it also comes with some standard force fields.
- Have a look at https://docs.openmm.org/7.6.0/userguide/application/02_running_sims.html#force-fields to see what sort of force fields you can use.

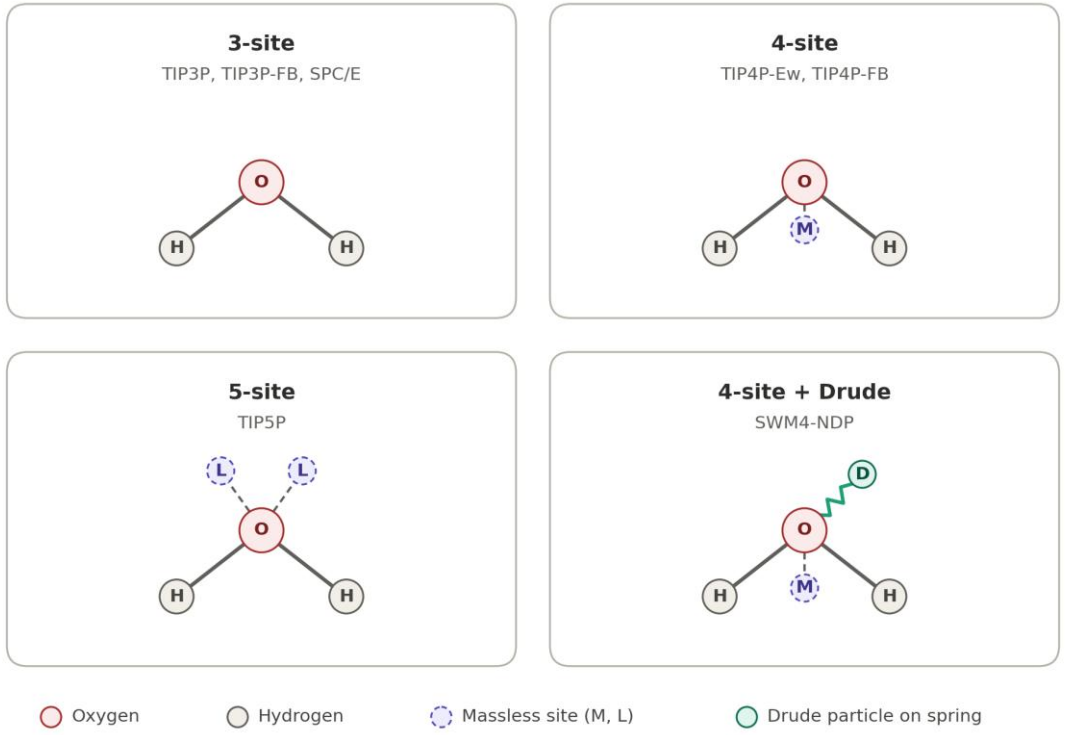
```
60  # load the pdf file and set the box vectors
61  pdb = app.PDBFile(PACKED_PDB)
62  topology = pdb.topology
63  topology.setPeriodicBoxVectors(
64      [mm.Vec3(3.105, 0, 0),
65       mm.Vec3(0, 3.105, 0),
66       mm.Vec3(0, 0, 3.105)] * unit.nanometer
67  )
68
69  # create the force field and add ex
70  forcefield = app.ForceField(...)
71  modeller = app.Modeller(pdb.topology, pdb.positions)
72  modeller.addExtraParticles(forcefield)
73  topology = modeller.topology
```

OpenMM

- Here are the water models that comes with OpenMM. They can be broadly grouped up into 3, 4, and 5-site models.
- TIP3P, TIP3P-FB, and SPC/E are 3-site water models.
- TIP4P, TIP4P-FB are 4-site water models.
- SWM4-NDP is a 4-site water model with a Drude particle.
- TIP5P is a 5-site water model.

File	Water Model
`tip3p.xml`	TIP3P water model[2]
`tip3pfb.xml`	TIP3P-FB water model[3]
`tip4pew.xml`	TIP4P-Ew water model[4]
`tip4pfb.xml`	TIP4P-FB water model[3]
`tip5p.xml`	TIP5P water model[9]
`spce.xml`	SPC/E water model[5]
`swm4ndp.xml`	SWM4-NDP water model[23]

https://docs.openmm.org/7.6.0/userguide/application/02_running_sims.html#water-models



OpenMM

- Here are the water models that comes with OpenMM. They can be broadly grouped up into 3, 4, and 5-site models.
- TIP3P, TIP3P-FB, and SPC/E are 3-site water models.
- TIP4P, TIP4P-FB are 4-site water models.
- SWM4-NDP is a 4-site water model with a Drude particle.
- TIP5P is a 5-site water model.
- Select the water model you want to use.

File	Water Model
`tip3p.xml`	TIP3P water model[2]
`tip3pfb.xml`	TIP3P-FB water model[3]
`tip4pew.xml`	TIP4P-Ew water model[4]
`tip4pfb.xml`	TIP4P-FB water model[3]
`tip5p.xml`	TIP5P water model[9]
`spce.xml`	SPC/E water model[5]
`swm4ndp.xml`	SWM4-NDP water model[23]

https://docs.openmm.org/7.6.0/userguide/application/02_running_sims.html#water-models

```
60  # load the pdf file and set the box vectors
61  pdb = app.PDBFile(PACKED_PDB)
62  topology = pdb.topology
63  topology.setPeriodicBoxVectors(
64      [mm.Vec3(3.105, 0, 0),
65       mm.Vec3(0, 3.105, 0),
66       mm.Vec3(0, 0, 3.105)] * unit.nanometer
67  )
68
69  # create the force field and add e      icles
70  forcefield = app.ForceField(...)
71  modeller = app.Modeller(pdb.topology, pdb.positions)
72  modeller.addExtraParticles(forcefield)
73  topology = modeller.topology
```

OpenMM

- In the next section, we create the OpenMM system. Here we also specify the method to sum the nonbonded interactions.
- In OpenMM the nonbonded interactions are the Lennard-Jones plus coulomb interactions. Here we use particle mesh Ewald for the long-range electrostatics and a cutoff of 0.9 nm for the direct space interactions.

```
75  # create the openmm system
76  system = forcefield.createSystem(
77      topology,
78      nonbondedMethod=app.PME,
79      nonbondedCutoff=0.9 * unit.nanometer,
80      rigidWater=...,
81  )
82  if not LIGHT_WATER:
83      transmute_water(system, topology, D_FRACTION)
84
85  # create and add the barostat
86  barostat_index = system.addForce(mm.MonteCarloBarostat(PRESSURE, TEMPERATURE, 25))
```

OpenMM

- In the next section, we create the OpenMM system. Here we also specify the method to sum the nonbonded interactions.
- In OpenMM the nonbonded interactions are the Lennard-Jones plus coulomb interactions. Here we use particle mesh Ewald for the long-range electrostatics and a cutoff of 0.9 nm for the direct space interactions.
- Here specify whether you want to run a rigid or flexible water models using `True` or `False`.

```
75  # create the openmm system
76  system = forcefield.createSystem(
77      topology,
78      nonbondedMethod=app.PME,
79      nonbondedCutoff=0.                ometer,
80      rigidWater=...,
81  )
82  if not LIGHT_WATER:
83      transmute_water(system, topology, D_FRACTION)
84
85  # create and add the barostat
86  barostat_index = system.addForce(mm.MonteCarloBarostat(PRESSURE, TEMPERATURE, 25))
```

OpenMM

- In the next section, we create the OpenMM system. Here we also specify the method to sum the nonbonded interactions.
- In OpenMM the nonbonded interactions are the Lennard-Jones plus coulomb interactions. Here we use particle mesh Ewald for the long-range electrostatics and a cutoff of 0.9 nm for the direct space interactions.
- Here specify whether you want to run a rigid or flexible water models using `True` or `False`.
- After the system is created, we change the water composition here.

```
75  # create the openmm system
76  system = forcefield.createSystem(
77      topology,
78      nonbondedMethod=app.PME,
79      nonbondedCutoff=0.9 * unit.nanometer,
80      rigidWater=...,
81  )
82  if not LIGHT_WATER:
83      transmute_water(system, topology, D_FRACTION)
84
85  # create and add the barostat
86  barostat_index = system.addForce(mm.MonteCarloBarostat(PRESSURE, TEMPERATURE, 25))
```

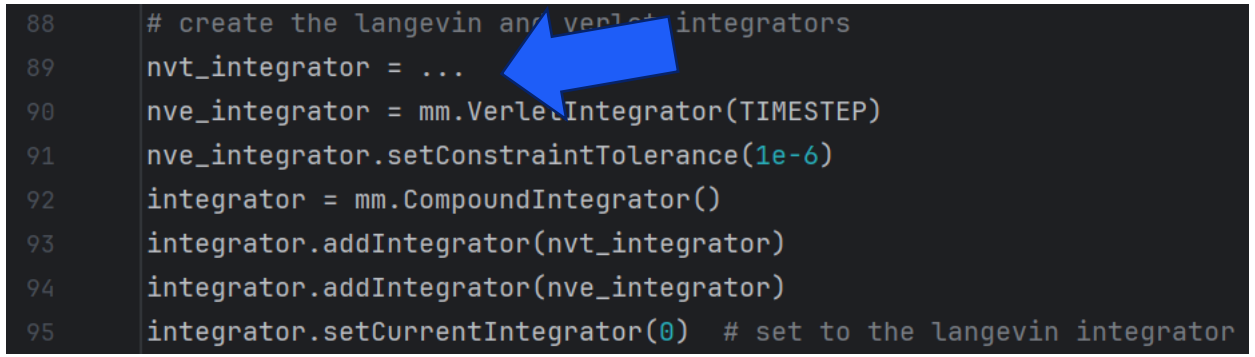
OpenMM

- Here we add a barostat to our simulation, it's the like the Monte-Carlo barostat you might have coded up yesterday if you had the chance.
- In the `MonteCarloBarostat` we input the target pressure and temperature as well as the Monte-Carlo volume change frequency. Here we use the same values as the default, so that there is a Monte-Carlo volume change every 25 time steps.

```
75  # create the openmm system
76  system = forcefield.createSystem(
77      topology,
78      nonbondedMethod=app.PME,
79      nonbondedCutoff=0.9 * unit.nanometer,
80      rigidWater=...,
81  )
82  if not LIGHT_WATER:
83      transmute_water(system, topology, D_FRACTION)
84
85  # create and add the barostat
86  barostat_index = system.addForce(mm.MonteCarloBarostat(PRESSURE, TEMPERATURE, 25))
```

OpenMM

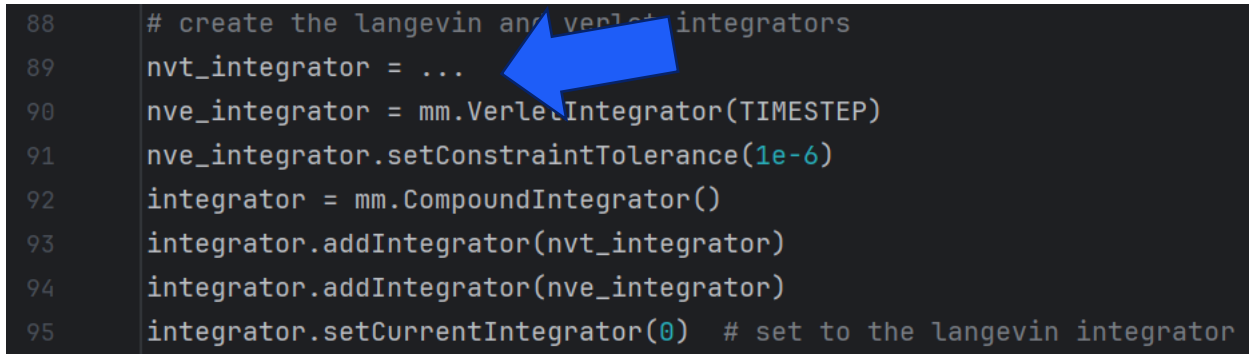
- Next, we create two integrators one for our NPT and NVT runs and one for our NVE run.
- Let's first look at the integrator we could use for our NPT and NVT runs. Go to https://docs.openmm.org/latest/userguide/theory/04_integrators.html.
- There are a few we can choose from. Remember that we need an integrator which samples at constant temperature. We need something other than the Verlet integrator which generates trajectories at constant energy.



```
88  # create the langevin and verlet integrators
89  nvt_integrator = ...
90  nve_integrator = mm.VerletIntegrator(TIMESTEP)
91  nve_integrator.setConstraintTolerance(1e-6)
92  integrator = mm.CompoundIntegrator()
93  integrator.addIntegrator(nvt_integrator)
94  integrator.addIntegrator(nve_integrator)
95  integrator.setCurrentIntegrator(0)  # set to the langevin integrator
```

OpenMM

- For our case we should choose between the `LangevinIntegator` and the `NoseHooverIntegrator`.
- Take a look at https://docs.openmm.org/latest/api-python/generated/openmm.openmm.LangevinIntegrator.html or https://docs.openmm.org/latest/api-python/generated/openmm.openmm.NoseHooverIntegrator.html to see how to add them to our script.



OpenMM

- Next, we create the Verlet integrator and the compound integrator and add both the NPT/NVT and Verlet integrators to it.
- The compound integrator allows use to switch between different integrators during our MD simulation.
- We start by setting the current integrator to the NPT/NVT integrator since we want to start the equilibration with an NPT run.

```
88    # create the langevin and verlet integrators
89    nvt_integrator = ...
90    nve_integrator = mm.VerletIntegrator(TIMESTEP)
91    nve_integrator.setConstraintTolerance(1e-6)
92    integrator = mm.CompoundIntegrator()
93    integrator.addIntegrator(nvt_integrator)
94    integrator.addIntegrator(nve_integrator)
95    integrator.setCurrentIntegrator(0)  # set to the langevin integrator
```



OpenMM

- In this part, of the script we create the OpenMM simulation and set starting positions and velocities.

```
97  # create the openmm simulation and set the positions and velocities
98  simulation = app.Simulation(topology, system, integrator)
99  simulation.context.setPositions(modeller.positions)
100 simulation.context.setVelocitiesToTemperature(TEMPERATURE)
101
102 # create some reporters which outputs some details of the simulation
103 simulation.reporters.append(app.StateDataReporter(
104     "equilibration.csv", REPORT_INTERVAL, step=True, time=True,
105     potentialEnergy=True, totalEnergy=True, temperature=True, volume=True,
106     density=True, separator=","))
107 simulation.reporters.append(app.StateDataReporter(
108     sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
109     density=True, speed=True))
```

OpenMM

- In this part, of the script we create the OpenMM simulation and set starting positions and velocities.
- In the next part we create the two reporters and add them to the simulations object. We create one which adds simulation information to a csv file and another which prints the information out to the terminal.

```
97  # create the openmm simulation and set the positions and velocities
98  simulation = app.Simulation(topology, system, integrator)
99  simulation.context.setPositions(modeller.positions)
100 simulation.context.setVelocitiesToTemperature(TEMPERATURE)
101
102 # create some reporters which outputs some details of the simulation
103 simulation.reporters.append(app.StateDataReporter(
104     "equilibration.csv", REPORT_INTERVAL, step=True, time=True,
105     potentialEnergy=True, totalEnergy=True, temperature=True, volume=True,
106     density=True, separator=","))
107 simulation.reporters.append(app.StateDataReporter(
108     sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
109     density=True, speed=True))
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.



```
111    # minimisation
112    print(f"Running OpenMM under the platform: {simulat      .context.getPlatform().getName()}")
113    simulation.minimizeEnergy(maxIterations=5000)
114
115    # NPT equilibration
116    print(f"Running NPT equilibration for: {NPT_STEPS} steps")
117    simulation.step(NPT_STEPS)
118
119    # NVT equilibration
120    print(f"Running NVT equilibration for: {NVT_STEPS} steps")
121    system.getForce(barostat_index).setFrequency(0)  # 0 disables the barostat
122    simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
123    simulation.step(NVT_STEPS)
124
125    # NVE equilibration
126    print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
127    integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
128    simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.
- Next, we run NPT to equilibrate the volume and temperatures.

```
111    # minimisation
112    print(f"Running OpenMM under the platform: {simulation.context.getPlatform().getName()}")
113    simulation.minimizeEnergy(maxIterations=5000)
114
115    # NPT equilibration
116    print(f"Running NPT equilibration for: {NPT_STEPS} steps")
117    simulation.step(NPT_STEPS)
118
119    # NVT equilibration
120    print(f"Running NVT equilibration for: {NVT_STEPS} steps")
121    system.getForce(barostat_index).setFrequency(0)  # 0 disables the barostat
122    simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
123    simulation.step(NVT_STEPS)
124
125    # NVE equilibration
126    print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
127    integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
128    simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.
- Next, we run NPT to equilibrate the volume and temperatures.
- Then we run NVT by turning the barostat off to further equilibrate the temperature at a constant volume.

```
111    # minimisation
112    print(f"Running OpenMM under the platform: {simulation.context.getPlatform().getName()}")
113    simulation.minimizeEnergy(maxIterations=5000)
114
115    # NPT equilibration
116    print(f"Running NPT equilibration for: {NPT_STEPS} steps")
117    simulation.step(NPT_STEPS)
118
119    # NVT equilibration
120    print(f"Running NVT equilibration for: {NVT_STEPS
121    system.getForce(barostat_index).setFrequency(0)  #  isables the barostat
122    simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
123    simulation.step(NVT_STEPS)
124
125    # NVE equilibration
126    print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
127    integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
128    simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.
- Next, we run NPT to equilibrate the volume and temperatures.
- Then we run NVT by turning the barostat off to further equilibrate the temperature at a constant volume.
- Then we switch to the NVE integrator to equilibrate at constant volume and energy.

```
111     # minimisation
112     print(f"Running OpenMM under the platform: {simulation.context.getPlatform().getName()}")
113     simulation.minimizeEnergy(maxIterations=5000)
114
115     # NPT equilibration
116     print(f"Running NPT equilibration for: {NPT_STEPS} steps")
117     simulation.step(NPT_STEPS)
118
119     # NVT equilibration
120     print(f"Running NVT equilibration for: {NVT_STEPS} steps")
121     system.getForce(barostat_index).setFrequency(0)  # 0 disables the barostat
122     simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
123     simulation.step(NVT_STEPS)
124
125     # NVE equilibration
126     print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
127     integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
128     simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Now we run our production run. First, we clear the reporters and add new ones including a DCD reporter which will output the trajectory into a DCD file.

```
# NVE production
print(f"Running NVE production for: {NVE_PROD_STEPS} steps")
# clear and create new reporters for the production stage
simulation.reporters.clear()
simulation.reporters.append(app.DCDReporter(OUTPUT_DCD_NAME, DCD_INTERVAL))
simulation.reporters.append(app.StateDataReporter(
    "production.csv", REPORT_INTERVAL, step=True, time=True,
    potentialEnergy=True, kineticEnergy=True, totalEnergy=True,
    temperature=True, volume=True, density=True, separator=","))
simulation.reporters.append(app.StateDataReporter(
    sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
    totalEnergy=True, speed=True, totalSteps=NVE_PROD_STEPS,
    progress=True))
# reset the time back to zero and run the production stage
simulation.context.setStepCount(0)
simulation.context.setTime(0 * unit.picosecond)
simulation.step(NVE_PROD_STEPS)

# Save the final structure. This is the topology file needed to the trajectory
# conversion with the DCD converter. We need to use this one and not the
# input PDB file since we might have added some virtual particles.
state = simulation.context.getState(getPositions=True, enforcePeriodicBox=True)
with open(OUTPUT_PDB_NAME, "w") as f:
    app.PDBFile.writeFile(simulation.topology, state.getPositions(), f)
```

OpenMM

- Now we run our production run. First, we clear the reporters and add new ones including a DCD reporter which will output the trajectory into a DCD file.
- Here we reset the step and time counter and run our simulation.

```
130  # NVE production
131  print(f"Running NVE production for: {NVE_PROD_STEPS} steps")
132  # clear and create new reporters for the production stage
133  simulation.reporters.clear()
134  simulation.reporters.append(app.DCDReporter(OUTPUT_DCD_NAME, DCD_INTERVAL))
135  simulation.reporters.append(app.StateDataReporter(
136      "production.csv", REPORT_INTERVAL, step=True, time=True,
137      potentialEnergy=True, kineticEnergy=True, totalEnergy=True,
138      temperature=True, volume=True, density=True, separator=","))
139  simulation.reporters.append(app.StateDataReporter(
140      sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
141      totalEnergy=True, speed=True, totalSteps=NVE_PROD_STEPS,
142      progress=True))
143  # reset the time back to zero and run the production st
144  simulation.context.setStepCount(0)
145  simulation.context.setTime(0 * unit.picosecond)
146  simulation.step(NVE_PROD_STEPS)
147
148  # Save the final structure. This is the topology file needed to the trajectory
149  # conversion with the DCD converter. We need to use this one and not the
150  # input PDB file since we might have added some virtual particles.
151  state = simulation.context.getState(getPositions=True, enforcePeriodicBox=True)
152  with open(OUTPUT_PDB_NAME, "w") as f:
153      app.PDBFile.writeFile(simulation.topology, state.getPositions(), f)
```

OpenMM

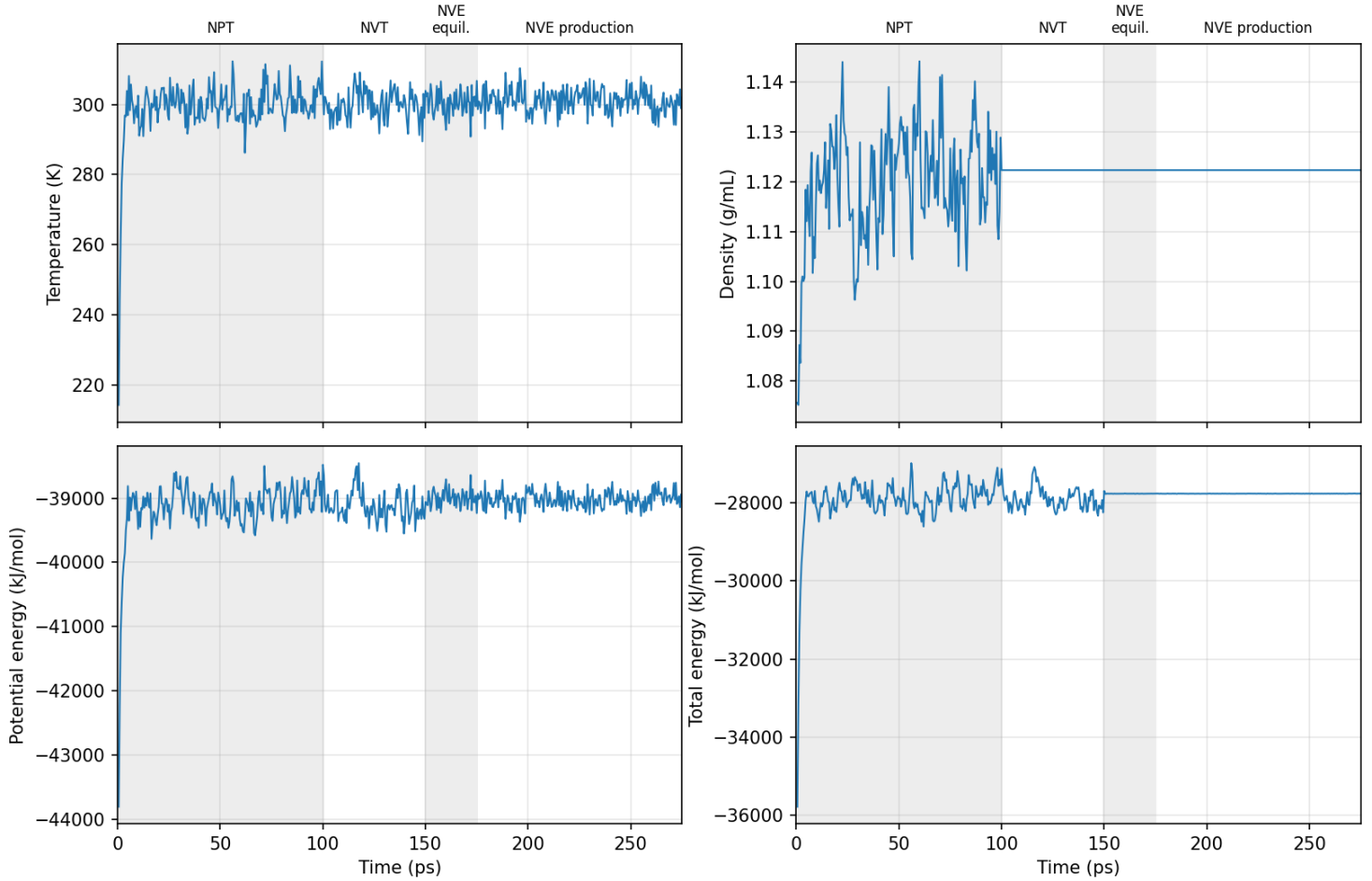
- Now we run our production run. First, we clear the reporters and add new ones including a DCD reporter which will output the trajectory into a DCD file.
- Here we reset the step and time counter and run our simulation.
- Here we write a PDB file for the final step. We need this to convert the file with MDANSE.
- Try running the openmm script once you have finished.

```
python run_water_box.py
```

```
130  # NVE production
131  print(f"Running NVE production for: {NVE_PROD_STEPS} steps")
132  # clear and create new reporters for the production stage
133  simulation.reporters.clear()
134  simulation.reporters.append(app.DCDReporter(OUTPUT_DCD_NAME, DCD_INTERVAL))
135  simulation.reporters.append(app.StateDataReporter(
136      "production.csv", REPORT_INTERVAL, step=True, time=True,
137      potentialEnergy=True, kineticEnergy=True, totalEnergy=True,
138      temperature=True, volume=True, density=True, separator=","))
139  simulation.reporters.append(app.StateDataReporter(
140      sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
141      totalEnergy=True, speed=True, totalSteps=NVE_PROD_STEPS,
142      progress=True))
143  # reset the time back to zero and run the production stage
144  simulation.context.setStepCount(0)
145  simulation.context.setTime(0 * unit.picosecond)
146  simulation.step(NVE_PROD_STEPS)
147
148  # Save the final structure. This is the topology file needed to the trajectory
149  # conversion with the DCD converter. We need to use this one and not the
150  # input PDB file since we might have added some virtual particles.
151  state = simulation.context.getState(getPositions=True, enforcePeriodicBox=True)
152  with open(OUTPUT_PDB_NAME, "w") as f:
153      app.PDBFile.writeFile(simulation.topology, state.getPositions(), f)
```

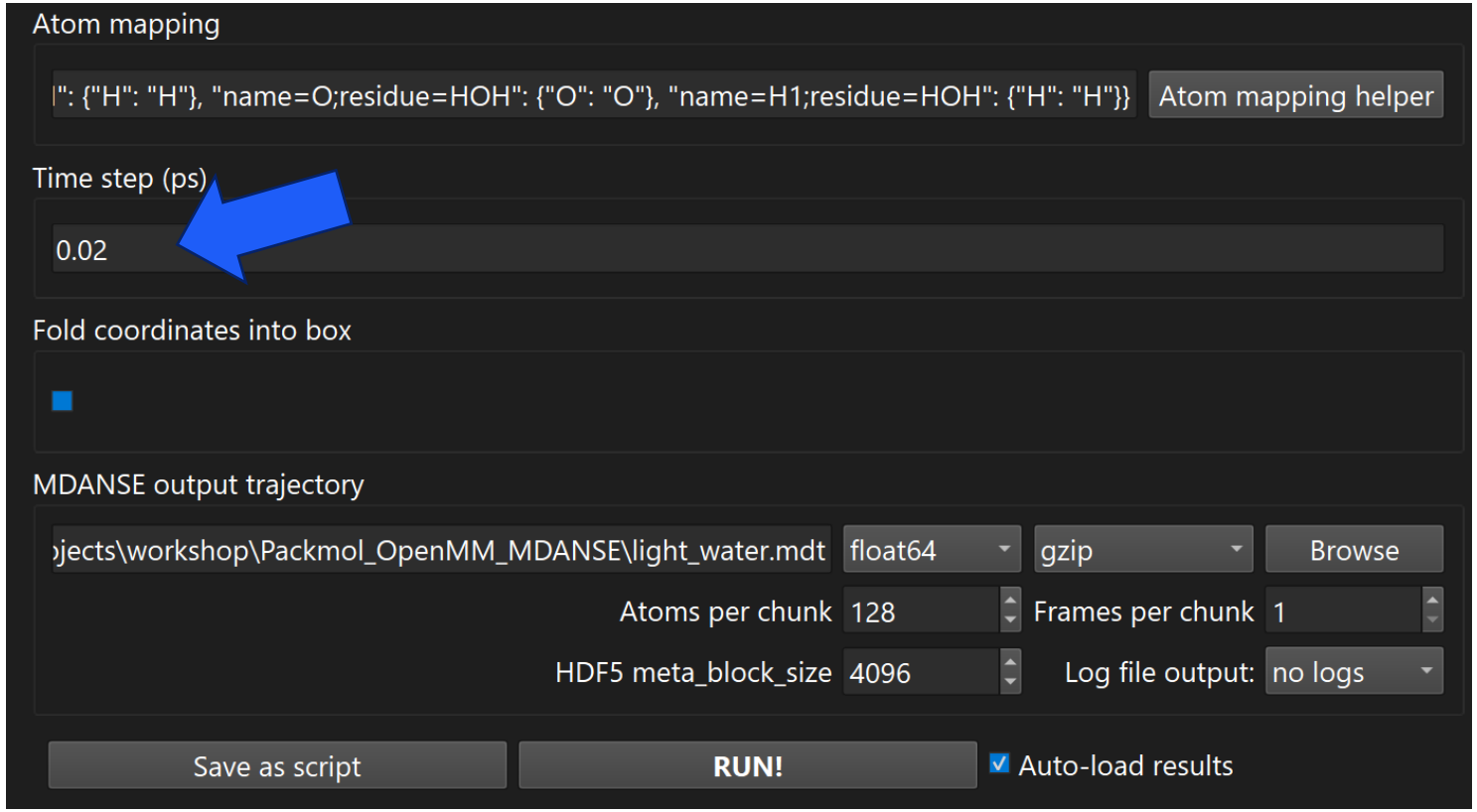
OpenMM

- There should be a few generate files, `equilibration.csv`, `production.csv`, and `production.csv`.
- In the files there is a script called `csv_plot.py` you can use this to plot the MD run information.
- If everything ran well, you should see a plot similar to the on one the right.
- Generally, we want to make sure we don't want to see any sort of drift in the results.



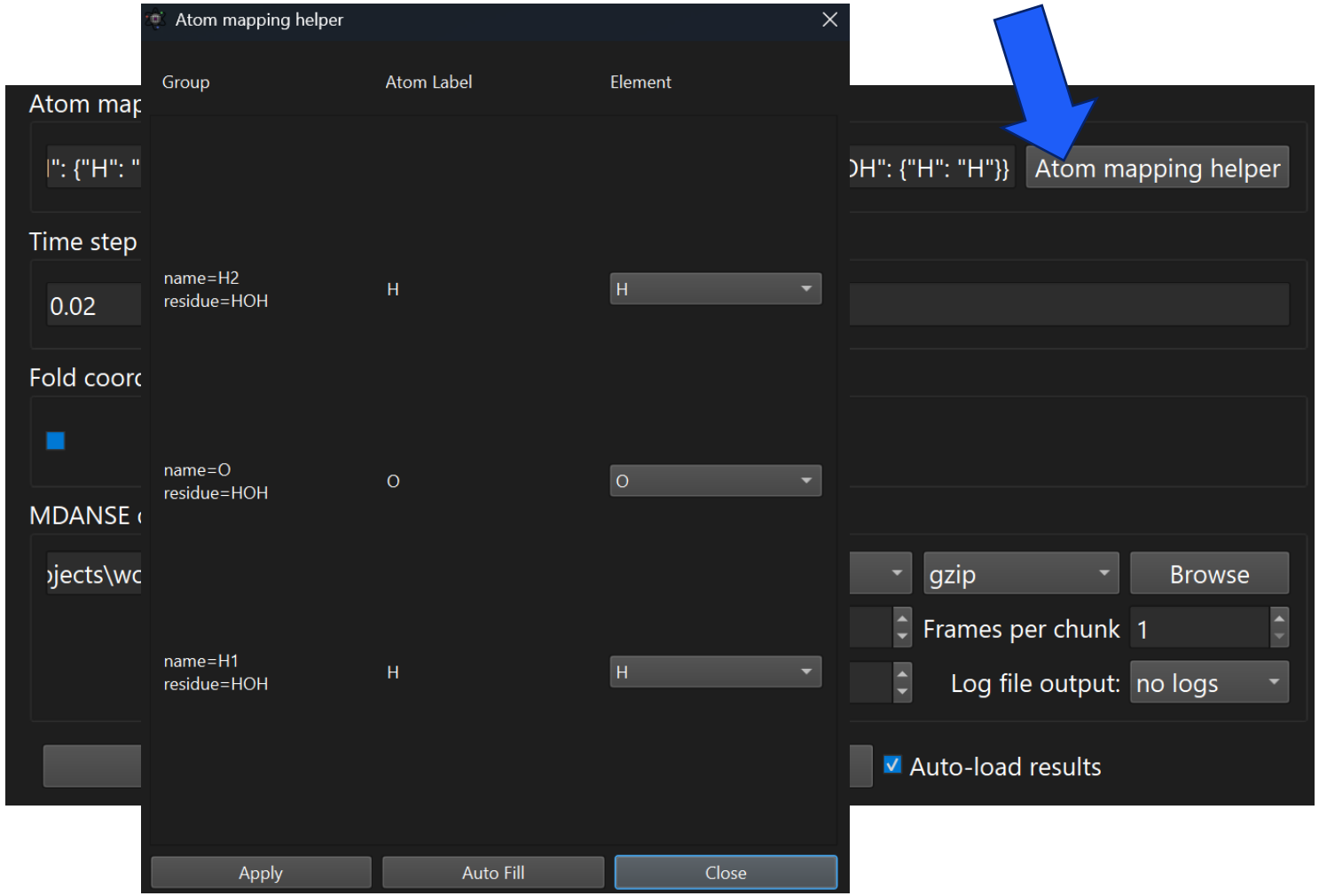
MDANSE

- Convert the DCD and PDB files to the MDANSE MDT format using the DCD converter. Make sure you set the correct time step.



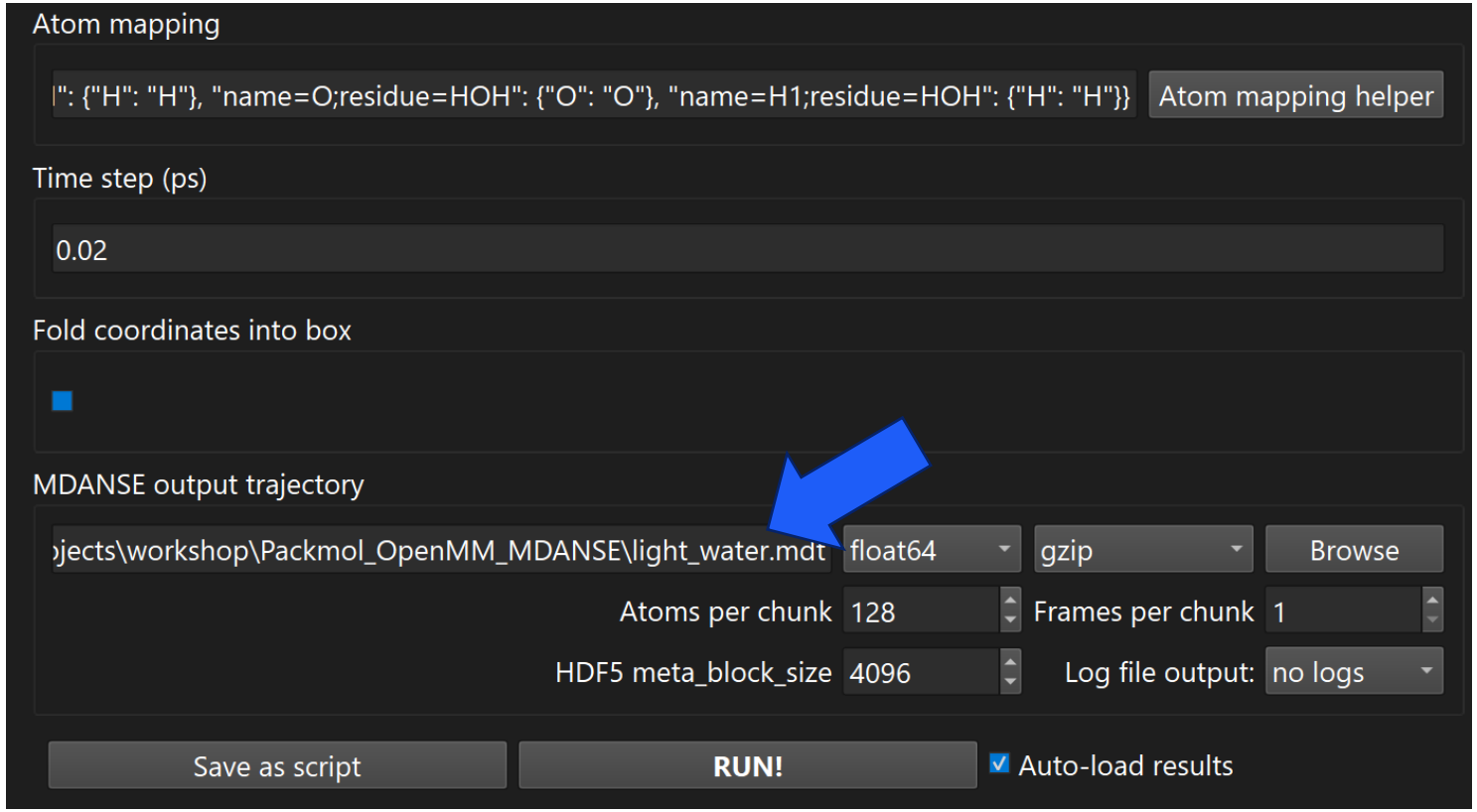
MDANSE

- Convert the DCD and PDB files to the MDANSE MDT format using the DCD converter. Make sure you set the correct time step.
- If used a water model with dummy atoms or etc, check that the correct atom types were assigned.



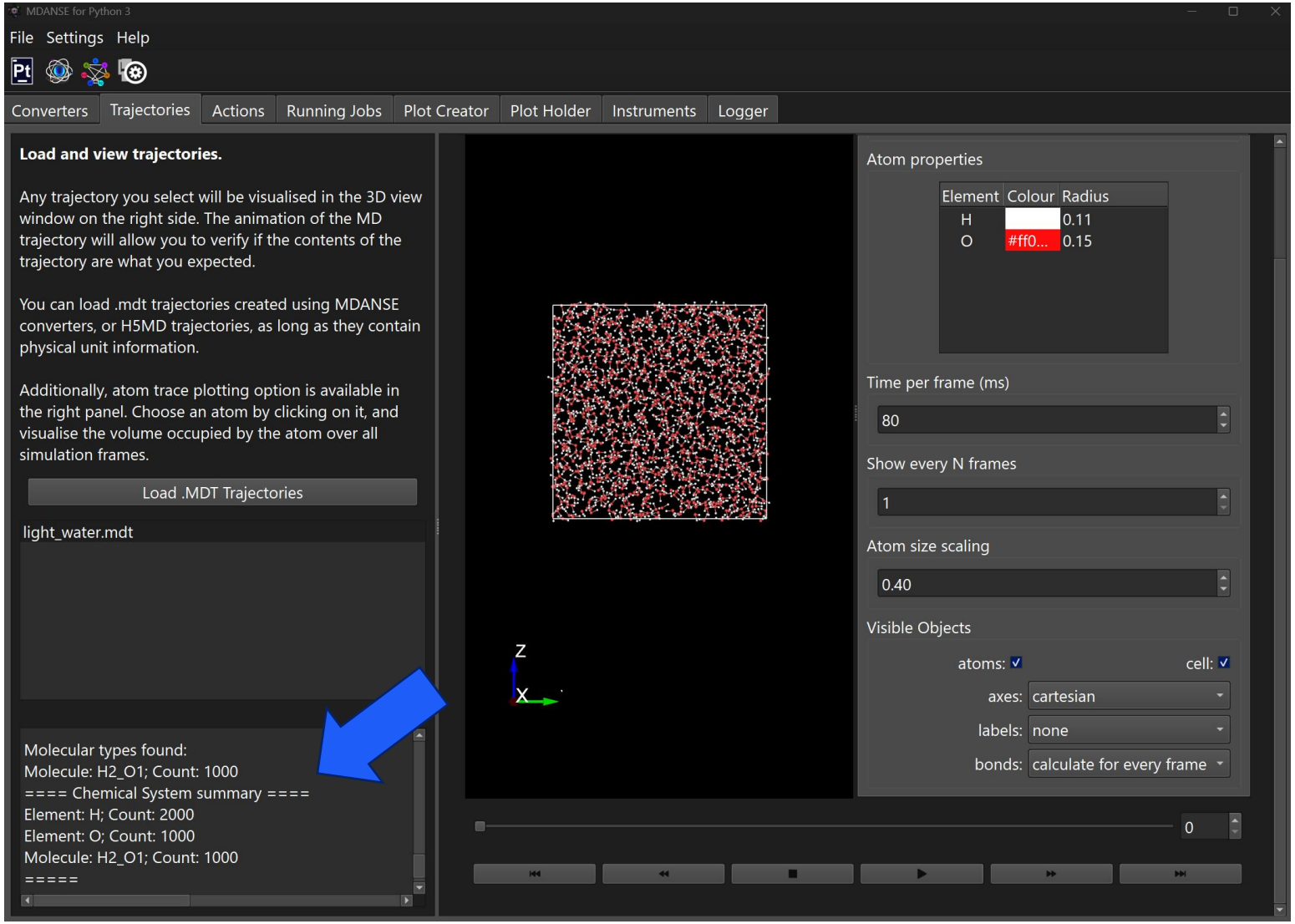
MDANSE

- Convert the DCD and PDB files to the MDANSE MDT format using the DCD converter. Make sure you set the correct time step.
- If used a water model with dummy atoms or etc, check that the correct atom types were assigned.
- We will need to create a total of three trajectories, light water, heavy water, and null water. Make sure to give them a good filename and path.



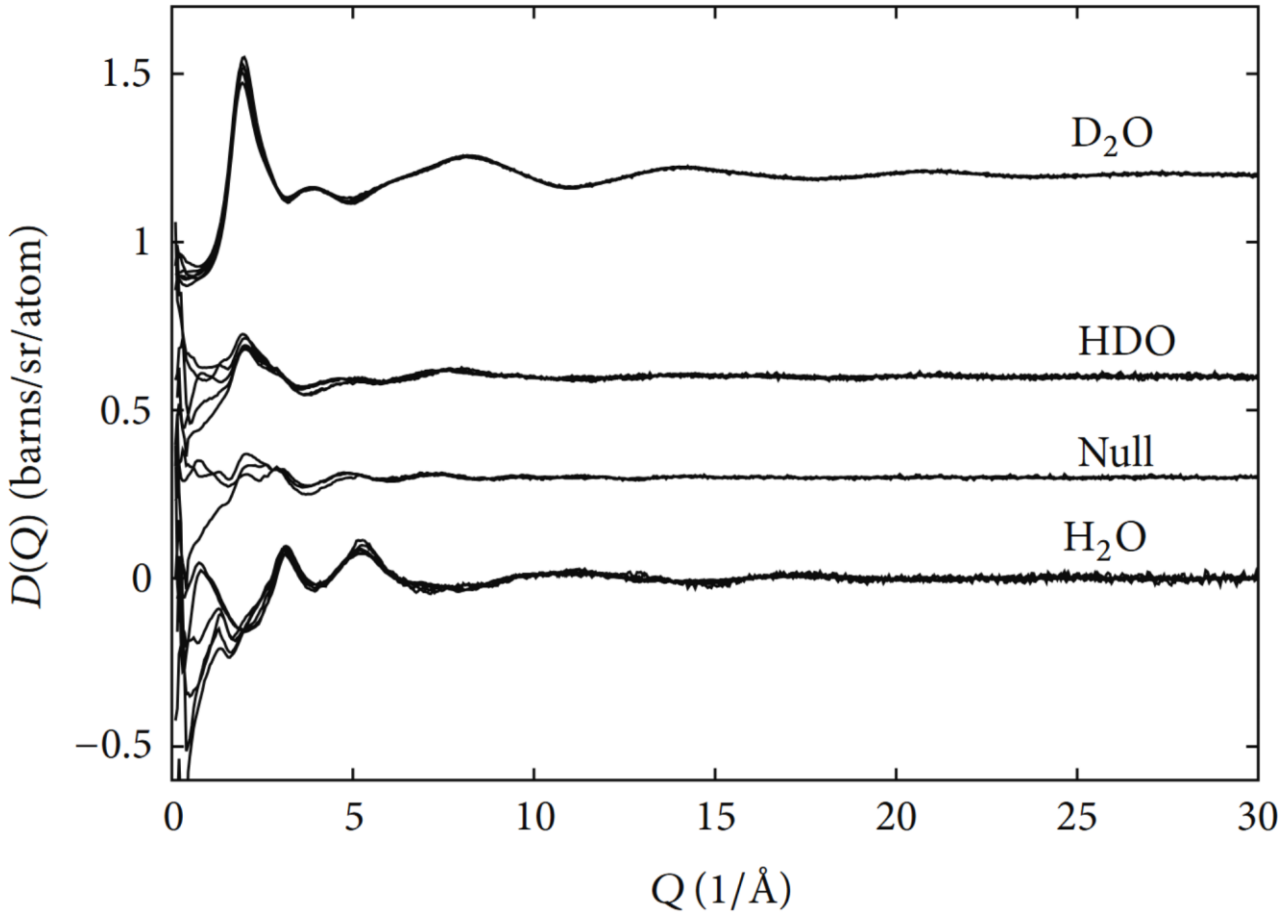
MDANSE

- Once complete view your trajectory in the trajectory tab and scroll down in the information box.
- Notice that MDANSE has grouped your atom into molecules. This is because the PDB file included some information which allows the DCD converter to group the atoms.



MDANSE

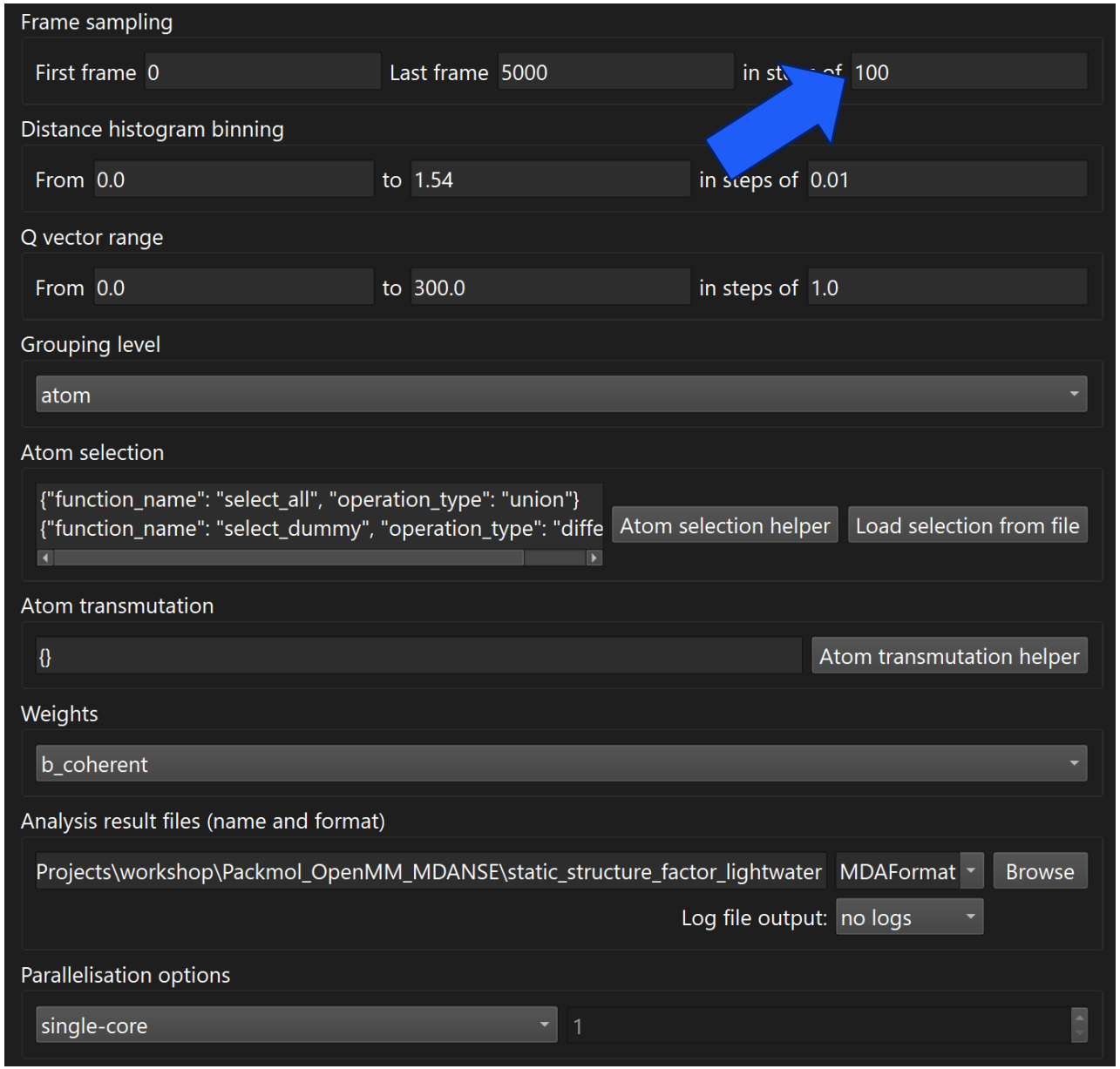
- Let's run a static structure factor calculation on our light water trajectory.



A. K. Soper, http://dx.doi.org/10.1155/2013/279463

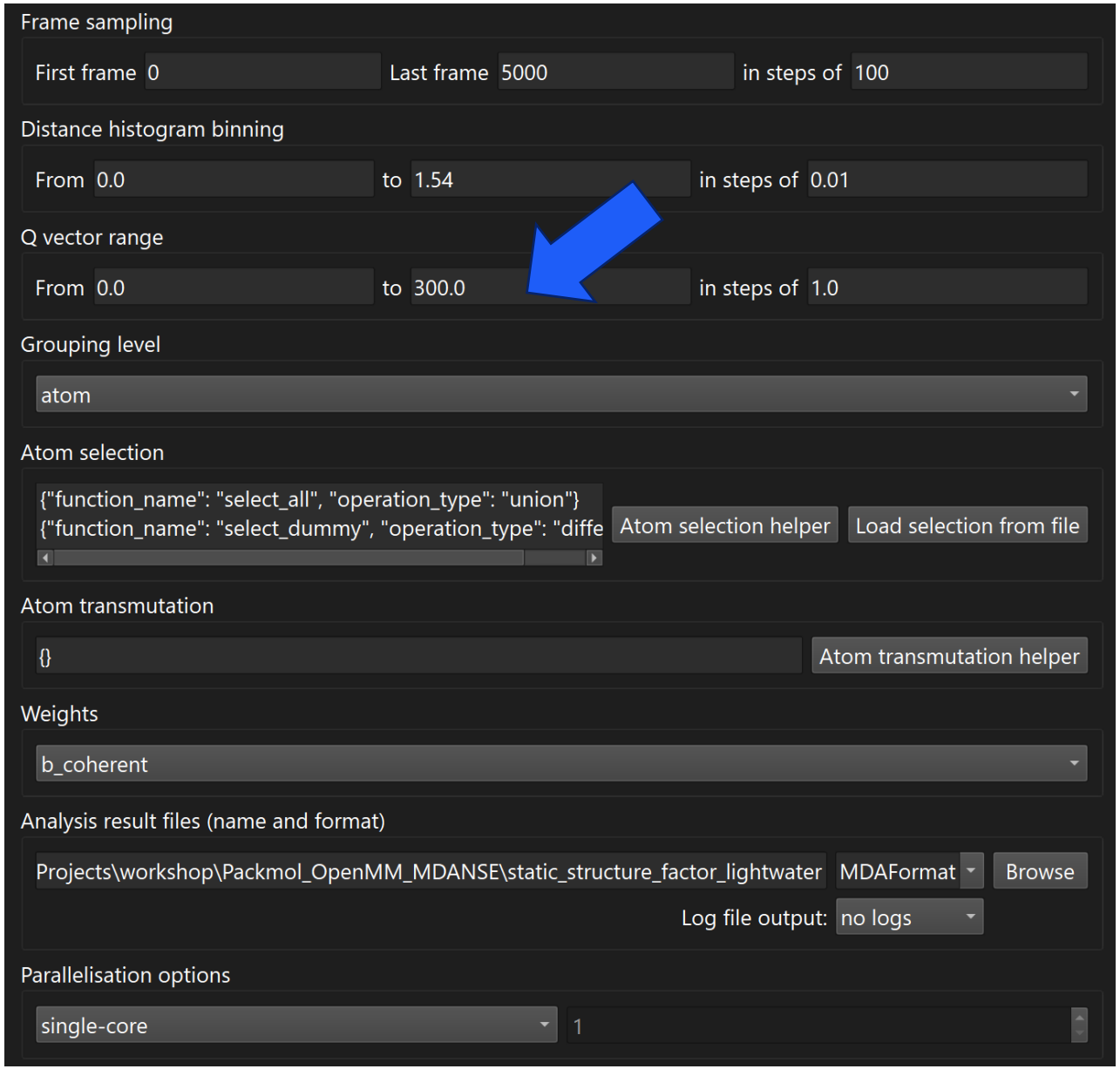
MDANSE

- Let's run a static structure factor calculation on our light water trajectory.
- With the static structure factor, the results are averaged over each input frame. To speed up the calculation we will only do for every 100th frame.



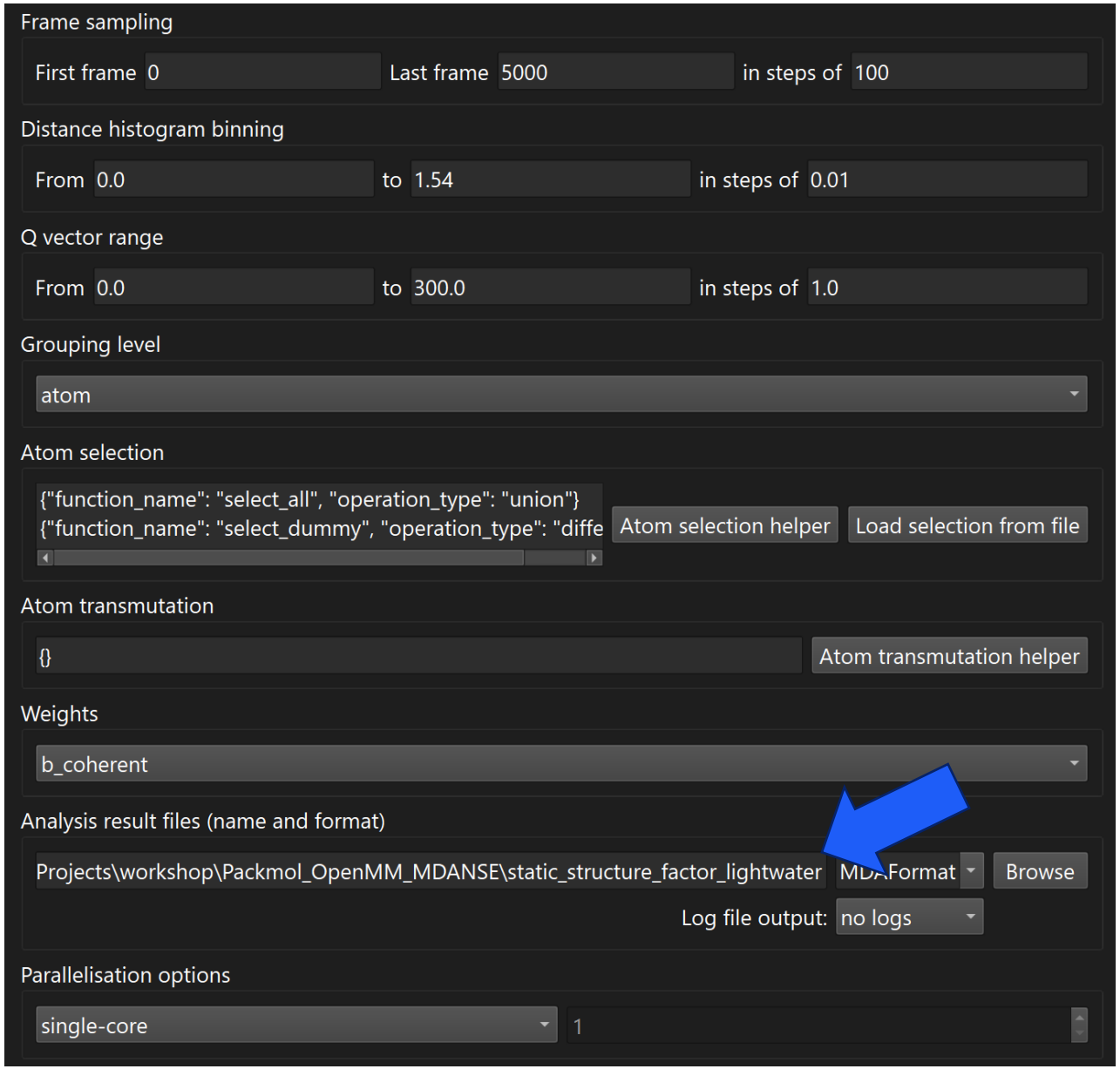
MDANSE

- Let's run a static structure factor calculation on our light water trajectory.
- With the static structure factor, the results are averaged over each input frame. To speed up the calculation we will only do for every 100th frame.
- The units in the q-vector range are in 1/nm we want to do something similar to the figure from Sopers paper so we set this to go up 300 1/nm.



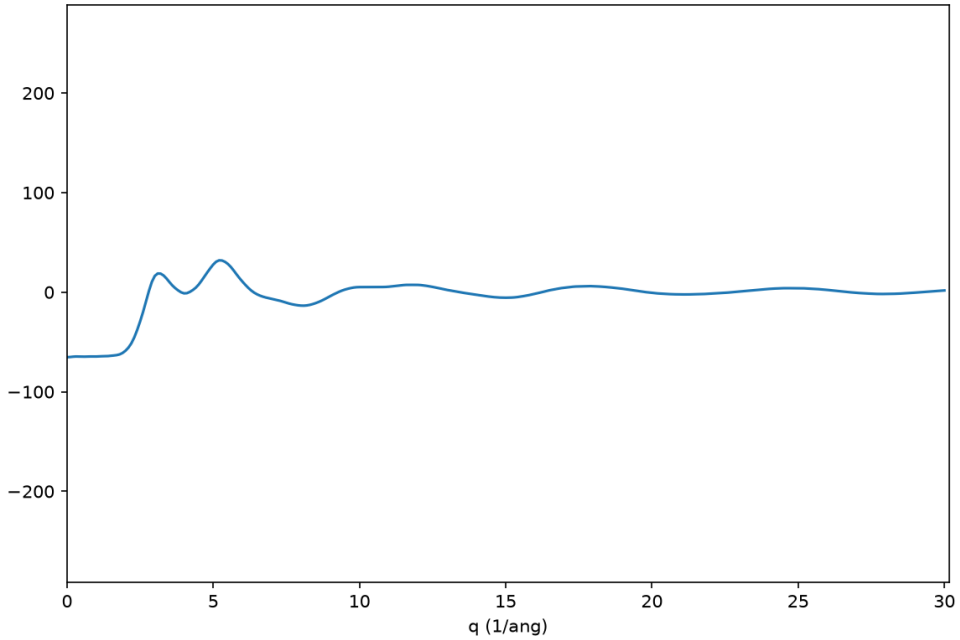
MDANSE

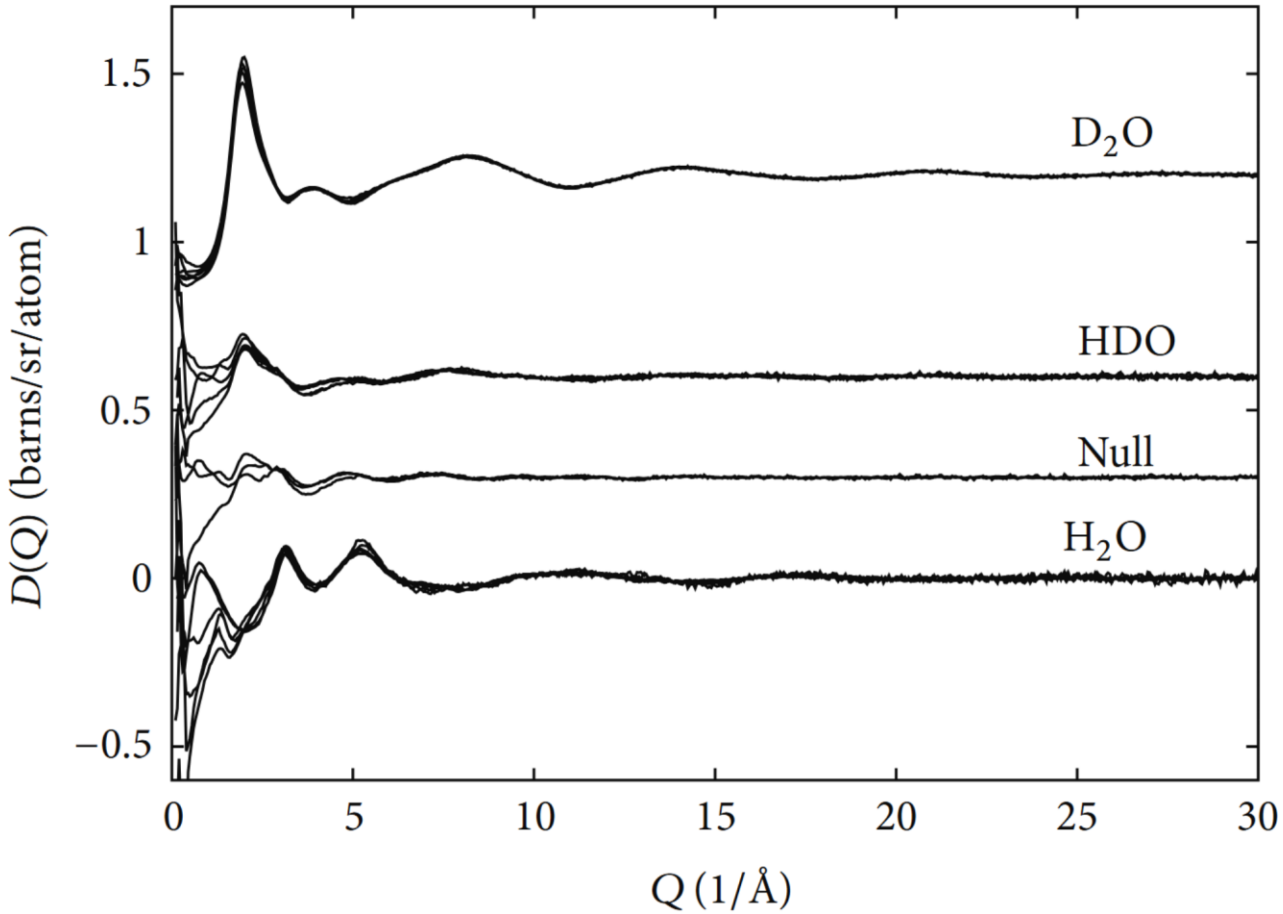
- Let's run a static structure factor calculation on our light water trajectory.
- With the static structure factor, the results are averaged over each input frame. To speed up the calculation we will only do for every 100th frame.
- The units in the q-vector range are in 1/nm we want to do something similar to the figure from Sopers paper so we set this to go up 300 1/nm.
- Change the filename and path to something suitable and hit run once you are ready.



MDANSE

- If everything went well, you should be able to plot the total static structure factor, and it should look similar to the result from Soper.

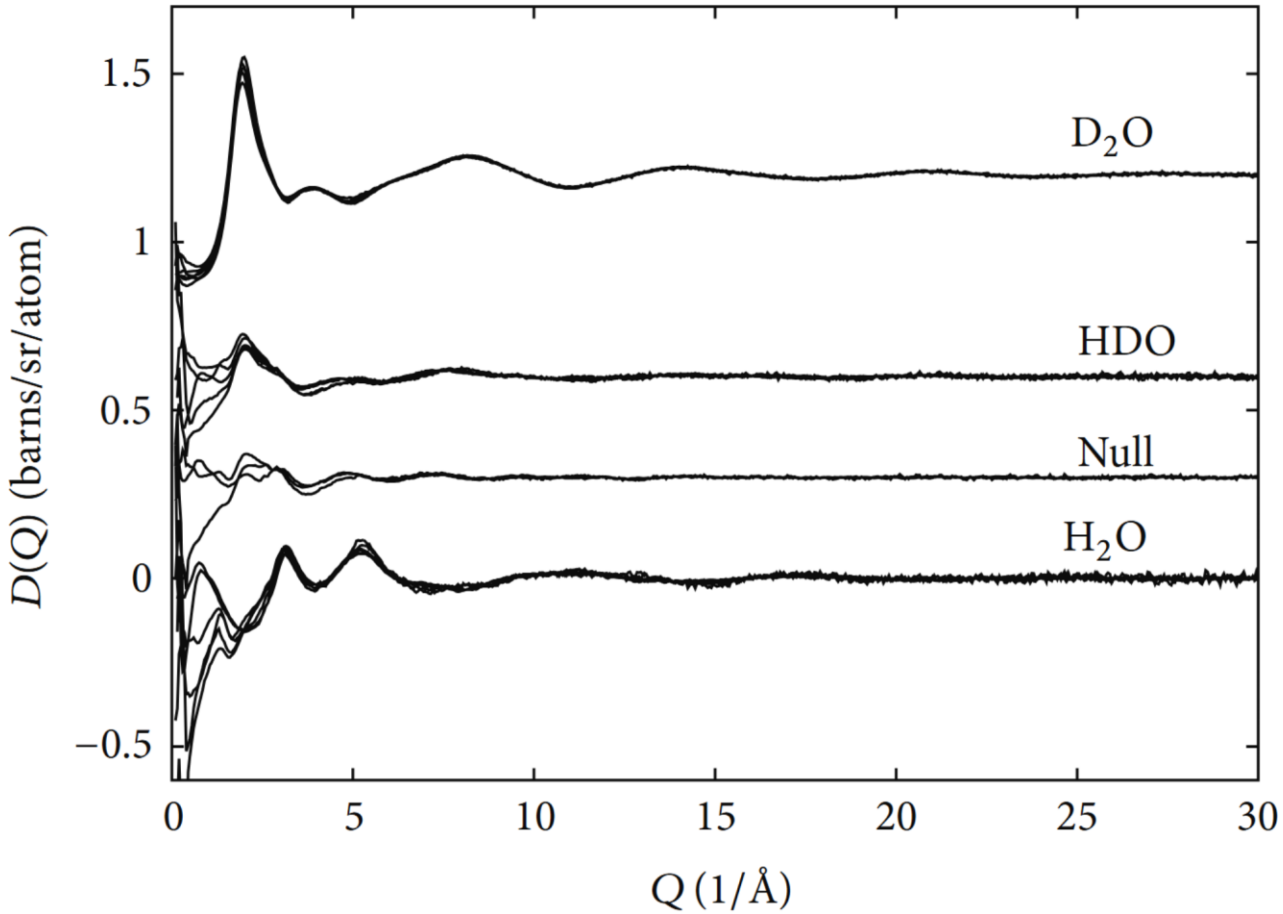




A. K. Soper, http://dx.doi.org/10.1155/2013/279463

MDANSE

- If everything went well, you should be able to plot the total static structure factor, and it should look similar to the result for Soper.
- If this went well, let's try to do the same thing again but for D2O and null water.



A. K. Soper, http://dx.doi.org/10.1155/2013/279463

OpenMM

- Let’s go back to our OpenMM script and run two more MD simulation.
- First the heavy water simulation. Just change the light water setting to False and fraction to 1.0.

```
9   rng = np.random.default_rng()
10
11  LIGHT_WATER = True
12  D_FRACTION = 0.0
13
14  PACKED_PDB = "water_box.pdb"
15  OUTPUT_DCD_NAME = ...
16  OUTPUT_PDB_NAME = ...
```

OpenMM

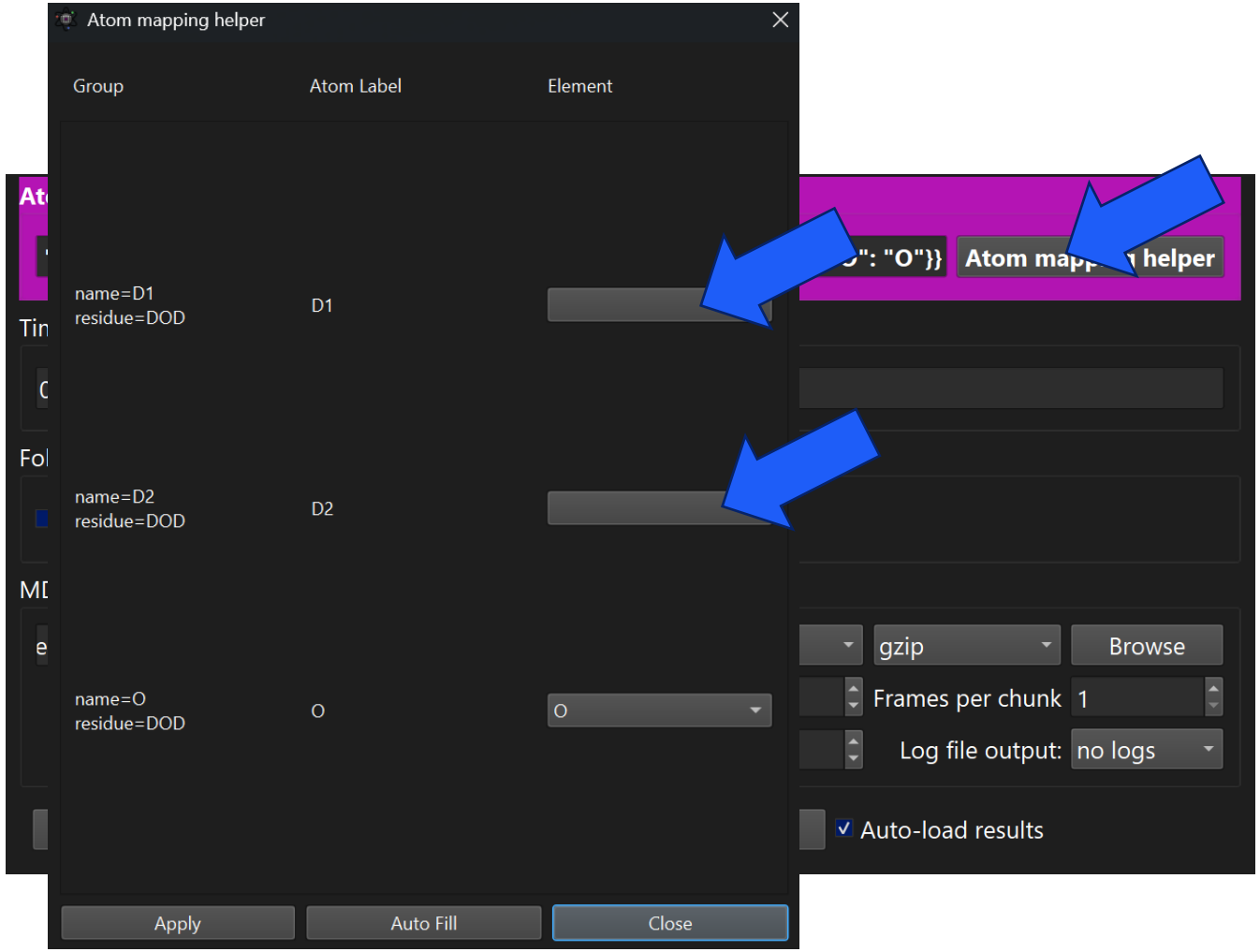
- Let’s go back to our OpenMM script and run two more MD simulation.
- First the heavy water simulation. Just change the light water setting to False and fraction to 1.0.
- Make sure you make a different name for the DCD and PDB output files.

```
9   rng = np.random.default_rng()
10
11  LIGHT_WATER = True
12  D_FRACTION = 0.0
13
14  PACKED_PDB = "water_box.pdb"
15  OUTPUT_DCD_NAME = ...
16  OUTPUT_PDB_NAME = ...
```



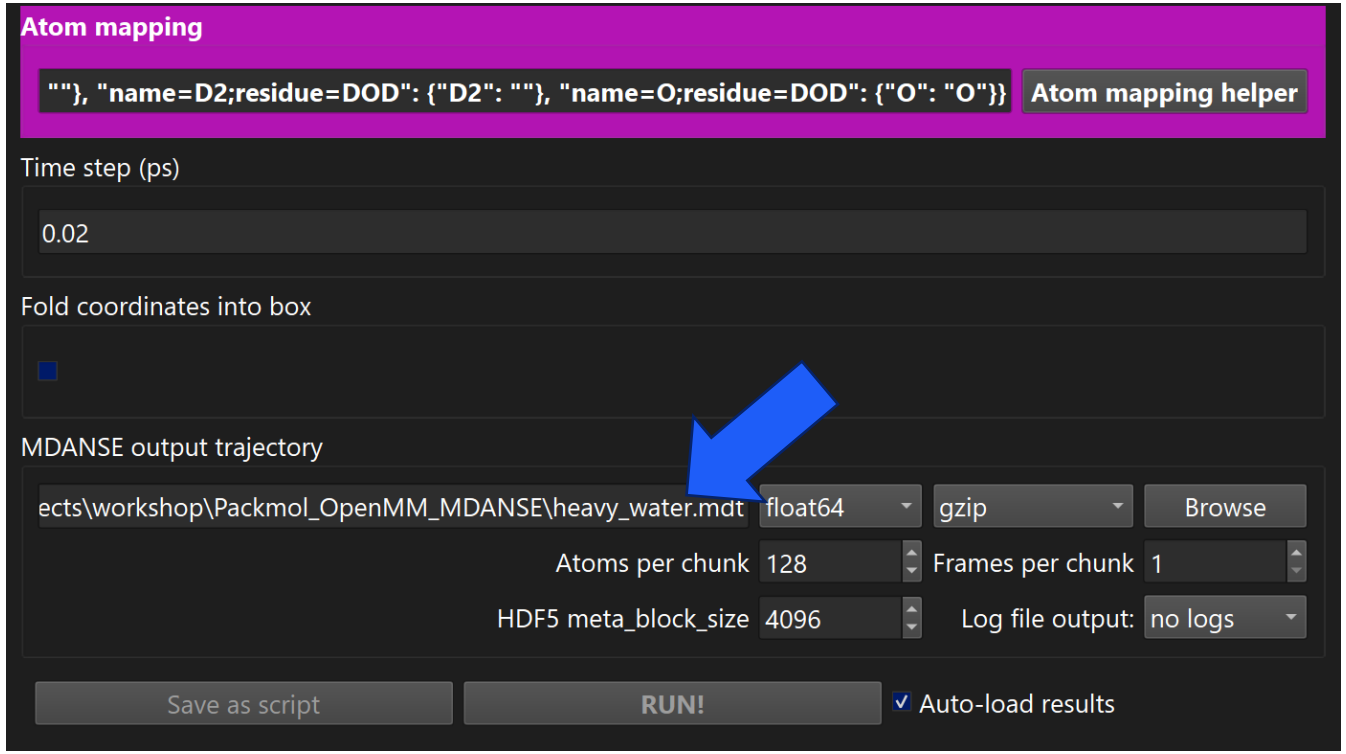
OpenMM

- Convert the file, in our OpenMM script we change the atom labels from H to D. We need to use the atom mapping to tell MDANSE to convert these to deuterium's H2.



OpenMM

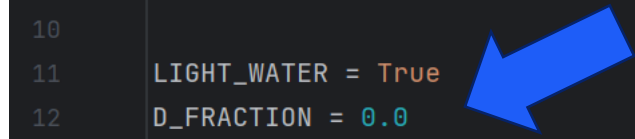
- Convert the file, in our OpenMM script we change the atom labels from H to D. We need to use the atom mapping to tell MDANSE to convert these to deuterium's H2.
- Give this another name to distinguish it from the light water trajectory.



OpenMM

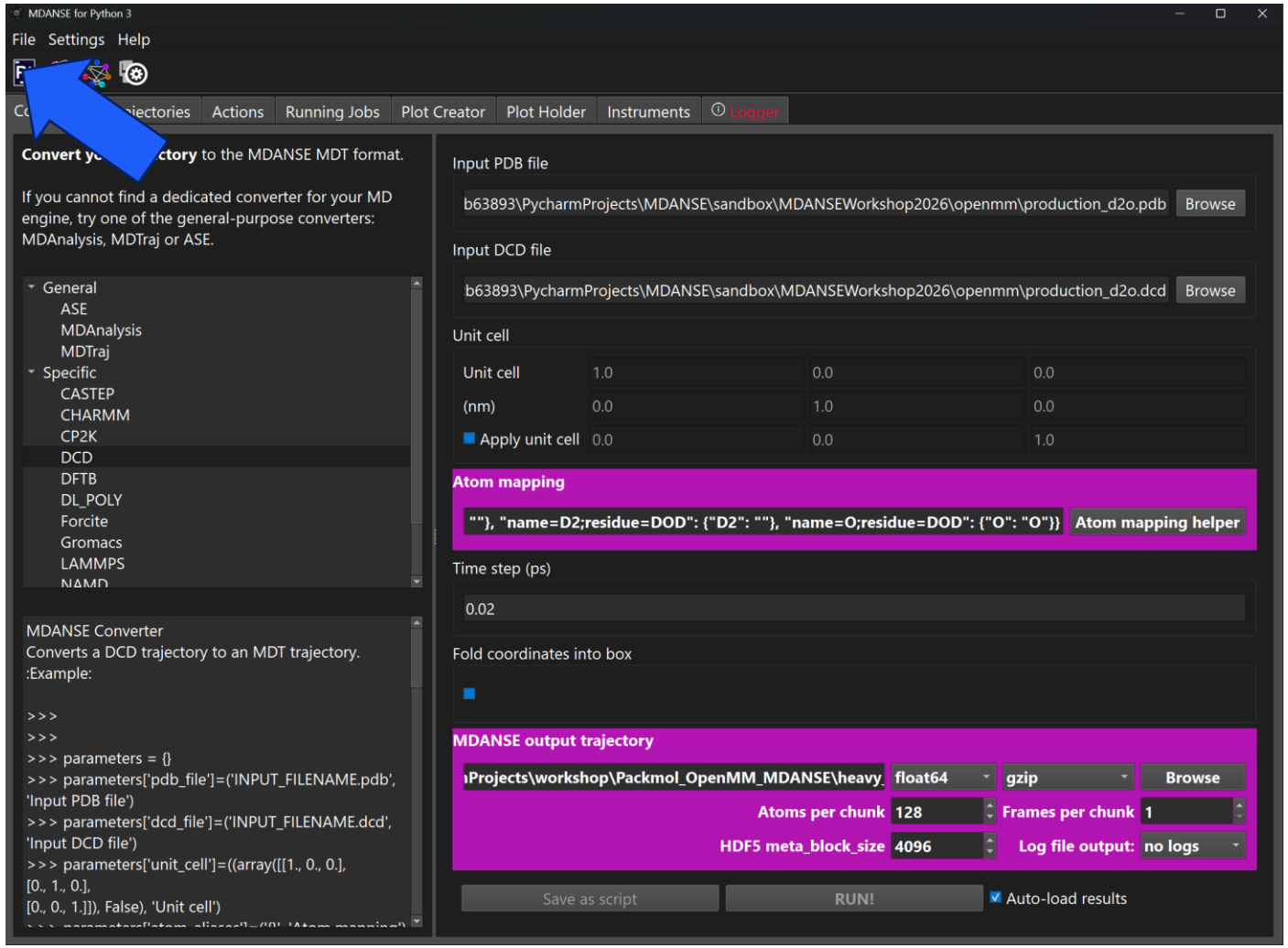
- Finally let's do it all again but for null water. First, we need to figure out the fraction of D2O that we need the D2O fraction.

```
9   rng = np.random.default_rng()
10
11  LIGHT_WATER = True
12  D_FRACTION = 0.0
13
14  PACKED_PDB = "water_box.pdb"
15  OUTPUT_DCD_NAME = ...
16  OUTPUT_PDB_NAME = ...
```



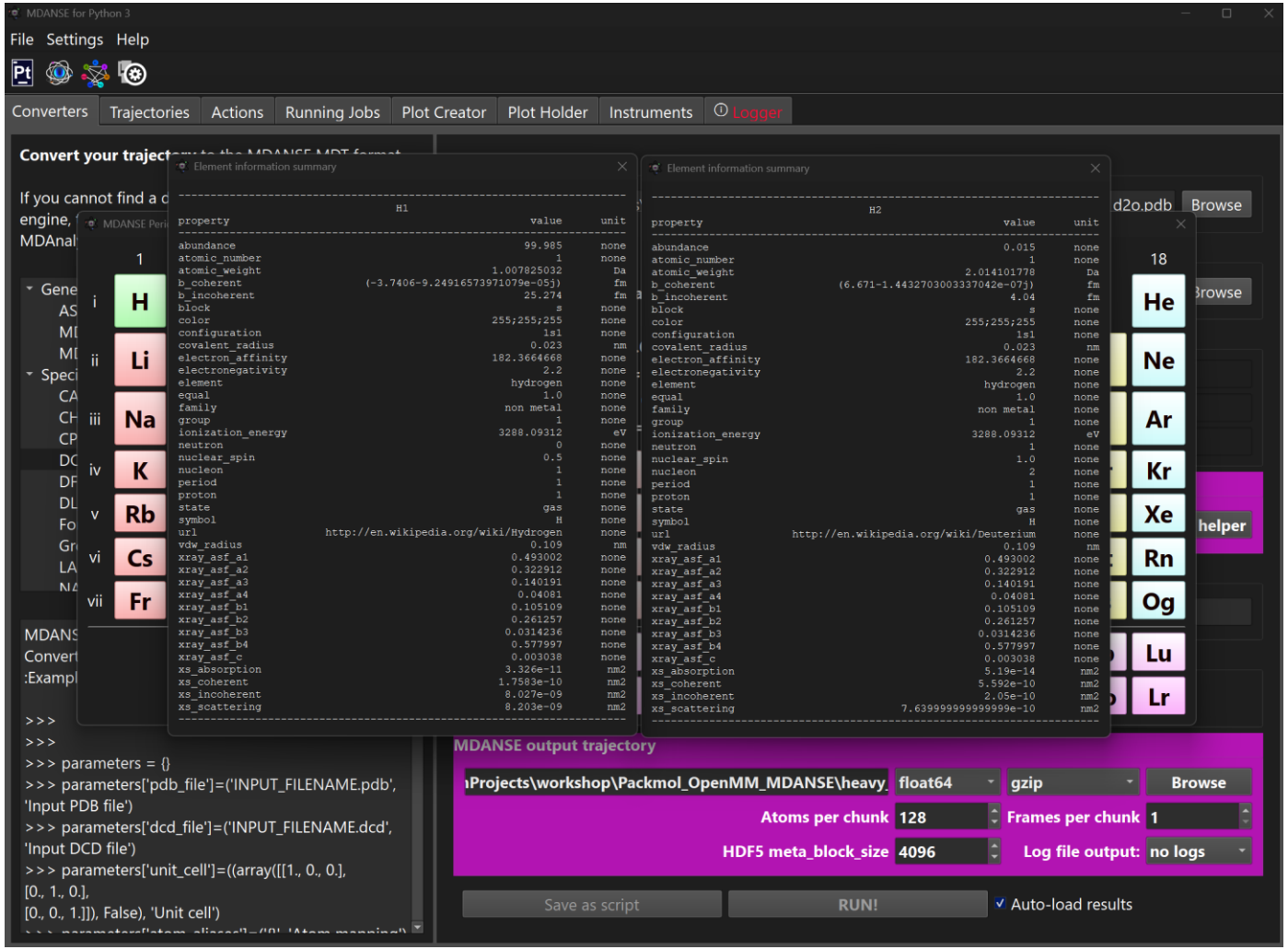
OpenMM

- Finally let's do it all again but for null water. First, we need to figure out the fraction of D2O that we need the D2O fraction.
- Go back to MDANSE and look at the periodic table.



OpenMM

- Finally let's do it all again but for null water. First, we need to figure out the fraction of D2O that we need the D2O fraction.
- Go back to MDANSE and look at the periodic table.
- Next click the hydrogen and then H1 to bring up the H1 element information. Bring up the H2 element information as well.



OpenMM

- Ignoring the imaginary part, H1 has a coherent scattering length of -3.7406 and H2 has a coherent scattering length of 6.671.
- Can you figure out the fraction of D2O we will need to create null water?

H1		
property	value	unit
abundance	99.985	none
atomic_number	1	none
atomic_weight	1.007825032	Da
b_coherent	(-3.7406-9.24916573971079e-05j)	fm
b_incoherent	25.274	fm

H2		
property	value	unit
abundance	0.015	none
atomic_number	1	none
atomic_weight	2.014101778	Da
b_coherent	(6.671-1.4432703003337042e-07j)	fm
b_incoherent	4.04	fm

OpenMM

- Ignoring the imaginary part, H1 has a coherent scattering length of -3.7406 and H2 has a coherent scattering length of 6.671.
- Can you figure out the fraction of D2O we will need to create null water?
- Once you have worked that out run the MD for null water and convert the trajectory.

H1

property	value	unit
abundance	99.985	none
atomic_number	1	none
atomic_weight	1.007825032	Da
b_coherent	(-3.7406-9.24916573971079e-05j)	fm
b_incoherent	25.274	fm

H2

property	value	unit
abundance	0.015	none
atomic_number	1	none
atomic_weight	2.014101778	Da
b_coherent	(6.671-1.4432703003337042e-07j)	fm
b_incoherent	4.04	fm

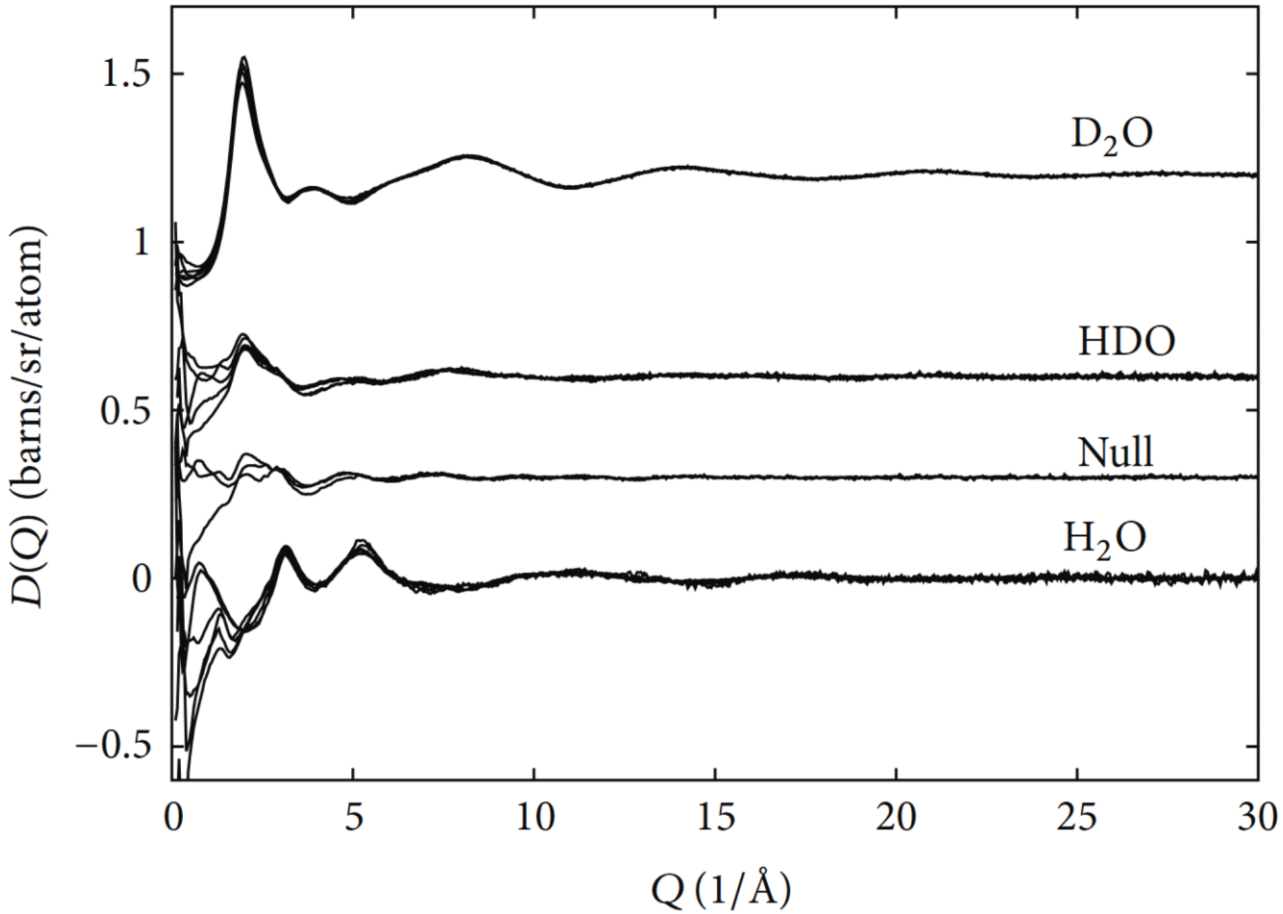
MDANSE

- Ignoring the imaginary part, H1 has a coherent scattering length of -3.7406 and H2 has a coherent scattering length of 6.671.
- Can you figure out the fraction of D2O we will need to create null water?
- Once you have worked that out run the MD for null water and convert the trajectory.
- Once the MD run is done convert with MDANSE, make sure you set the atom types correctly. H to H1 for example.



MDANSE

- Run the static structure factor calculation with heavy and null water.
- Do your results look similar to those from Soper?
- Look at the null water result in more detail. First, plot the H2O and H1O contribution to the scattering function what do they sum to?
- Next compare the H1H1, H2H2, and H1H2 what do they sum to?
- Finally plot the total and OO scattering function.
- Note, ignore very small values of q, as there will be a significant error associated to them due to our finite simulation box size.



A. K. Soper, http://dx.doi.org/10.1155/2013/279463